

04. Dashboard Market Survey and Advanced UX/UI Research for the Planned Site

Expanded research and design report for the planned private medium-term investment platform

Purpose	Translate dashboard market research into the updated site specification
Use case	Private medium-term portfolio decision support for one owner-operator
Design thesis	Decision-first, chart-first, risk-visible, replayable, and operationally transparent experience
Research basis	Public product documentation, public market positioning, open-source patterns, and prior FinRL-X architecture synthesis

This report studies the dashboard patterns and UX/UI principles that are most relevant to the planned site and converts them into direct design guidance. The report does not assume that the site should imitate a single benchmark product. Instead, it examines what integrated research platforms, portfolio analytics products, chart-first environments, and operational dashboards each contribute to a robust private decision workspace.

The result is a recommendation for a dashboard architecture that helps the user answer five core questions: what is the current recommendation, why was it generated, how risky is it, how trustworthy is it, and what changed since the previous decision cycle.

A key design insight from the prior analysis is that the true gap versus FinRL-X is not the number of models but the absence of a unified decision framework.

A second key insight is that the product should evolve from a multi-engine analytics dashboard into a weight-centric portfolio decision system with explicit Selection, Allocation, Timing, and Portfolio-Level Risk Overlay stages.

Market Landscape

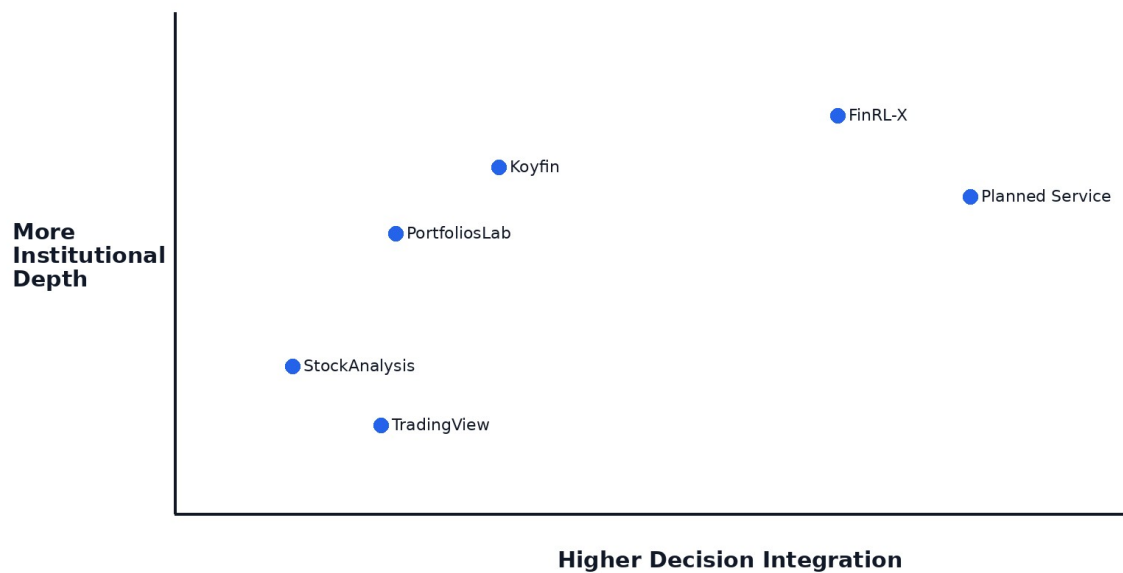


Figure 1. Market landscape and intended positioning of the planned site.

Site Information Architecture

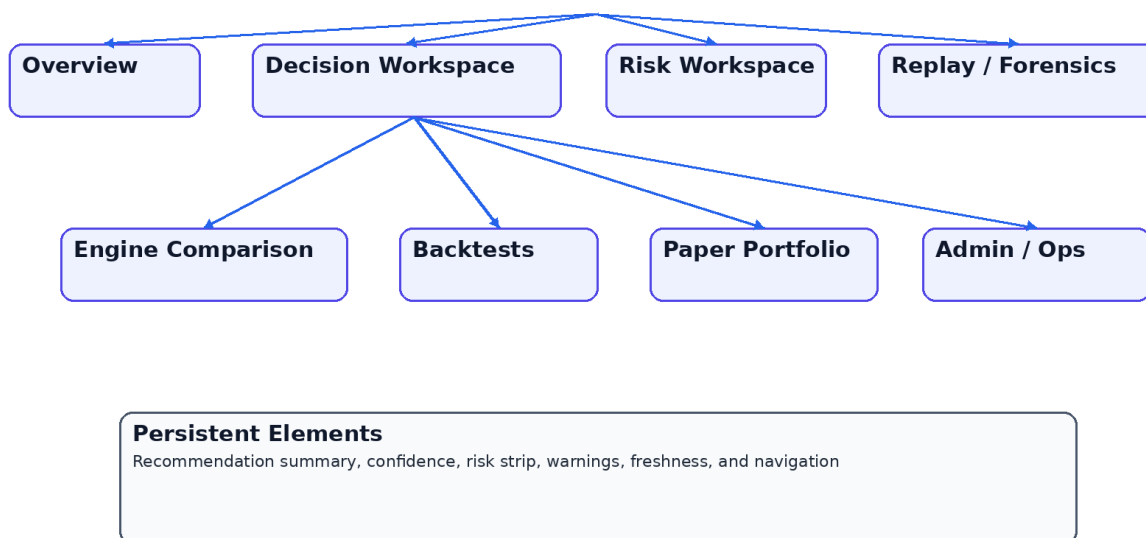


Figure 2. Proposed information architecture.

Admin / Ops Workspace

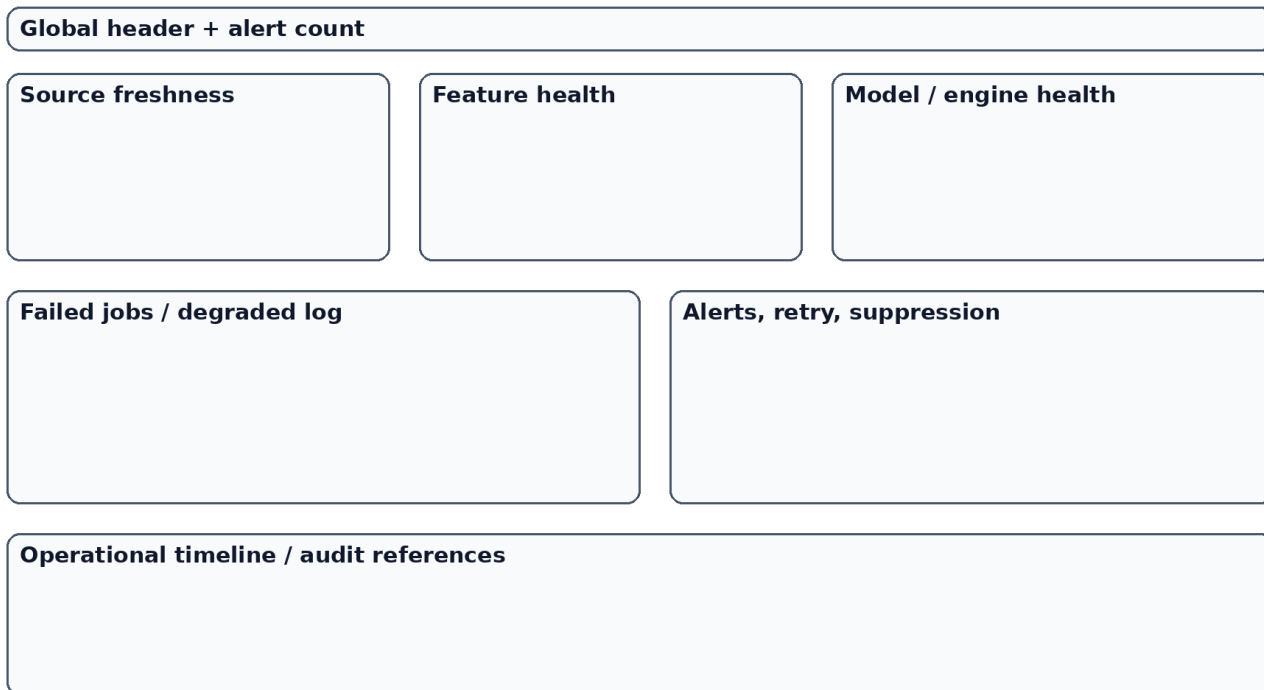


Figure 6. Admin / Ops concept.

Competitor Capability Heatmap

	News/Text	Portfolio	Technical	Scenario	Replay	Ops/Trust
Koyfin	2	3	2	2	1	1
PortfoliosLab	1	3	1	2	1	1
TradingView	1	1	3	1	1	1
StockAnalysis	2	1	1	1	1	1
Planned Service	3	3	3	3	3	3

Figure 7. Competitive capability heatmap.

Dashboard class	Strengths	Weaknesses	Recommended use in the planned site
Integrated research platforms	Breadth of context, data adjacency, cross-asset perspective	Can become broad but shallow	Borrow context integration and reduce clutter with tighter hierarchy
Portfolio analytics products	Strong risk, benchmark, correlation, and allocation views	Often weaker on narrative or text-driven intelligence	Borrow portfolio-centric interpretation and benchmark awareness
Chart-first trading tools	Strong interactive charts, overlays, alerts, comparison	Can overweight visual exploration over decision synthesis	Use charting as the narrative canvas for recommendation changes
Admin / ops dashboards	Strong pipeline truth, freshness, and incident visibility	Can feel too technical for end users	Expose operational truth in a curated, productized form

Executive Interpretation of the Dashboard Market

Dashboard products in finance often optimize for either breadth or immediacy, but rarely for coherent decision architecture. The planned site must optimize for coherent decision architecture.

Because the product is private and expert-facing, it can carry more structure than consumer tools as long as that structure is cleanly organized and tied to specific user questions.

The strongest market lesson is that users benefit when research context and portfolio context appear together. The weakest market pattern is the undifferentiated wall of metrics that leaves synthesis to the user.

Executive Interpretation of the Dashboard Market must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Lessons from Integrated Research Platforms

Platforms such as Koyfin validate that users value research, charting, portfolio awareness, and reporting in one place. The lesson is not to mimic any one interface pixel for pixel. The lesson is that integration reduces cognitive switching costs.

For the planned site, integrated context is especially important because a recommendation informed by text, sentiment, technical signals, and portfolio state must feel like one decision, not like four separate stories that happen to appear on the same page.

Lessons from Integrated Research Platforms must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade

interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Lessons from Portfolio Analytics Tools

Portfolio analytics tools validate the enduring importance of benchmark comparison, diversification analysis, drawdown interpretation, and concentration awareness.

The planned site should therefore treat portfolio implications as first-class. Recommendation quality must always be visible together with the portfolio-level consequences of acting on that recommendation.

Lessons from Portfolio Analytics Tools must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Lessons from Chart-First Products

Chart-first platforms validate the importance of fast, legible, and interactive time-series exploration. For advanced users, charts are often the fastest path to intuition about trend, volatility, correlation, and anomaly.

For this platform, the chart should become the main explanatory surface. It should not merely display price. It should show recommendation changes, timing blocks, risk cuts, and benchmark context.

Lessons from Chart-First Products must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Open-Source Dashboard Patterns and Operational Interfaces

Open-source dashboards often reveal useful truths that consumer products hide: data freshness, failed jobs, stale signals, and partially degraded states.

Those truths matter for the planned site because operational degradation is itself part of the user's trust model. If a source is stale or a model failed, the recommendation should say so clearly.

Open-Source Dashboard Patterns and Operational Interfaces must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Implications of the FinRL-X Decision Framework for UX

The uploaded notes clarified that the system should not continue as a collection of side-by-side engines. The UX implication is that the dashboard must be organized around a recommendation object and a policy pipeline.
[Integrated from uploaded notes]

This means the screen system must reveal Selection, Allocation, Timing, and Risk Overlay explicitly or at least traceably, rather than burying them beneath abstract 'signals'.

Implications of the FinRL-X Decision Framework for UX must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Information Architecture for the Planned Site

The site should separate Overview, Main Decision Workspace, Engine Comparison, Risk, Replay, Backtests, Paper Portfolio, and Admin / Ops into distinct workspaces.

This separation is not bureaucracy. It is how the product preserves clarity while supporting advanced capability.

Information Architecture for the Planned Site must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Main Decision Workspace Design

The main dashboard should be a three-zone desktop workspace with controls on the left, chart and decision breakdown in the center, and recommendation plus trust context on the right.

The central design objective is to compress a large amount of information into a stable and interpretable layout.

Main Decision Workspace Design must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Risk Workspace Design

Risk should be a dedicated workspace because risk is not a subordinate annotation. It is a policy layer and a product concern.

The risk screen should show exposures, concentration, stress context, active constraints, and risk history, including how the overlay changed the recommendation.

Risk Workspace Design must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Replay and Forensics UX

Replay must be user-facing because it answers one of the platform's most important trust questions: how did the system arrive at this recommendation?

The replay screen should show stage chronology, before/after states, narrative explanation, and audit references.

Replay and Forensics UX must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Backtests and Paper Portfolio as Validation Surfaces

Validation screens should not feel detached from the main product. They should share language, terminology, and schema with the recommendation system.

Backtests answer how the policy behaved historically under assumptions; paper portfolio answers how it behaves in simulated forward operation.

Backtests and Paper Portfolio as Validation Surfaces must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Admin and Ops UX

Admin / Ops must present source freshness, feature integrity, engine health, publication state, and incident history in separate domains.

This protects the user from false certainty and makes degraded states visible rather than accidental.

Admin and Ops UX must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into

one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Mobile UX Strategy

Mobile should be recommendation-first and triage-oriented. It should emphasize what changed, whether risk shifted, whether warnings exist, and what the current recommendation is.

Deep comparison matrices and long replay exploration should be secondary or collapsible on mobile rather than forced into parity.

Mobile UX Strategy must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Design-System Direction

The visual system should feel premium, calm, and analytical rather than loud or gamified.

Strong color should be reserved for status, risk, warnings, and emphasis. Stable spacing, readable tables, and chart clarity matter more than decorative novelty.

Design-System Direction must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Explicit Site-Spec Updates Derived from the Research

1. Reframe the site as a modular portfolio decision system, not as a generic dashboard.
2. Define recommendation, confidence, risk, freshness, and change narrative as the top-level information block on Overview and Dashboard.
3. Update the dashboard to reflect Selection, Allocation, Timing, and Risk Overlay either explicitly or through traceable drill-down structure.
4. Create dedicated workspaces for Risk, Replay, Backtests, Paper Portfolio, and Admin / Ops.
5. Define mobile as triage-first rather than parity-first.
6. Require all relevant API responses to include warning states, freshness semantics, and audit references where applicable.

References and Input Integration

This document incorporates the two uploaded architecture notes as controlling design assumptions. The first note argued that the real gap is the lack of a unified decision framework rather than a shortage of models, while the

second argued for a shift toward a weight-centric modular portfolio recommendation system. Those insights directly shaped the dashboard architecture and site-structure guidance in this report.

External sources reflected in the analysis include official positioning and feature pages from Koyfin and TradingView, along with public materials around portfolio analytics and dashboard patterns. Relevant current source evidence includes Koyfin's statement that it helps investors research markets and analyze portfolios in one place, and TradingView's emphasis on charting, screeners, and portfolio analysis features.

Appendix A — Scenario Control Patterns and Control-Surface Design

This supplementary section is included to ensure that the dashboard report translates market observations into implementable site-spec guidance. A robust dashboard for this product should preserve recommendation hierarchy, explain policy-stage effects, keep risk and trust visible, and maintain operational honesty. That means component placement, progressive disclosure, mobile prioritization, and admin visibility all need explicit specification rather than vague direction.

Appendix B — Component-Level UX Heuristics for the Planned Dashboard

Appendix C — Error, Warning, Empty, and Degraded States